

# COMP3438 System Programming

Unix File System

# Course Organization

## Part I: Unix System Programming (Device Driver Development)

Character Device Driver  
Development

Introduction to Block  
Device Driver

Overview of Device Driver Development

Process

File System

Overview of Unix Systems Programming

## Part II: Compiler Design

Syntax Analysis

Lexical Analysis

Overview of Compiler Design

Overview of the Subject

# Unix File System

What is a file in Unix?

How many types of files?

How are the files in a file system structured?

How is a file represented in memory and disk?

How to find the file using its name?

How to access files from a Unix program?

# Unix File System

File system provides abstractions of naming, storage, and access of files.

A file is a container of some information: data, program.

In Unix, devices (disks, tapes, CD ROMs, screens, keyboards, printers, mice, etc.) are also treated as files, so as to provide a unified and device independent interface to applications.

# How to handle devices?

OS provides **system calls** to the programmer for performing control and I/O to devices. These system calls are handled by **device drivers**, which hide the details of device operation and protect the devices from unauthorized use.

Some OS provides specific system calls for each type of supported devices.

In contrast, in Unix, disk files and other devices (except network devices) are named and accessed in the same way as data files.

- Unix provides a uniform device interface (called file descriptors).
- Allow uniform access to most devices through file system calls – `open`, `close`, `read`, `write`, etc.

# Types of files in Unix

**Regular** file: an ordinary data/program file on disk – contains bytes of data/program organized into a linear array;

**Special** file: a file representing a device – located in the `/dev` directory

**Block** special file: devices transferring info in blocks or chunks, just like disks, CD ROM.

**Character** special file: devices transferring info in stream of bytes (can also be randomly accessed, if the driver allows), e.g., keyboard, printer.

**FIFO** special file: used for inter-process communications (e.g. pipe).

**Directories:** provided to allow names (not physical locations) of files to be used.

User gives a file name and Unix makes a translation to the location of the physical file – done via directories.

# Difference between regular and directory file

Contents: data/program vs file info

Operations: what can be done and who can do them

# The `ls -l` command

For each file you can see ten characters before the user and group owner. The first character tells us the type of the file.

| first character | file type        |
|-----------------|------------------|
| -               | normal file      |
| d               | directory        |
| l               | symbolic link    |
| p               | named pipe       |
| b               | block device     |
| c               | character device |
| s               | socket           |

# The `ls -l` command

Next 9 characters represent access right assignment for user, group, and all.

| <b>permission</b> | <b>on a file</b>          | <b>on a directory</b>        |
|-------------------|---------------------------|------------------------------|
| r (read)          | read file contents (cat)  | read directory contents (ls) |
| w (write)         | change file contents (vi) | create files in (touch)      |
| x (execute)       | execute the file          | enter the directory (cd)     |

# What are the differences between a regular file and a device file?

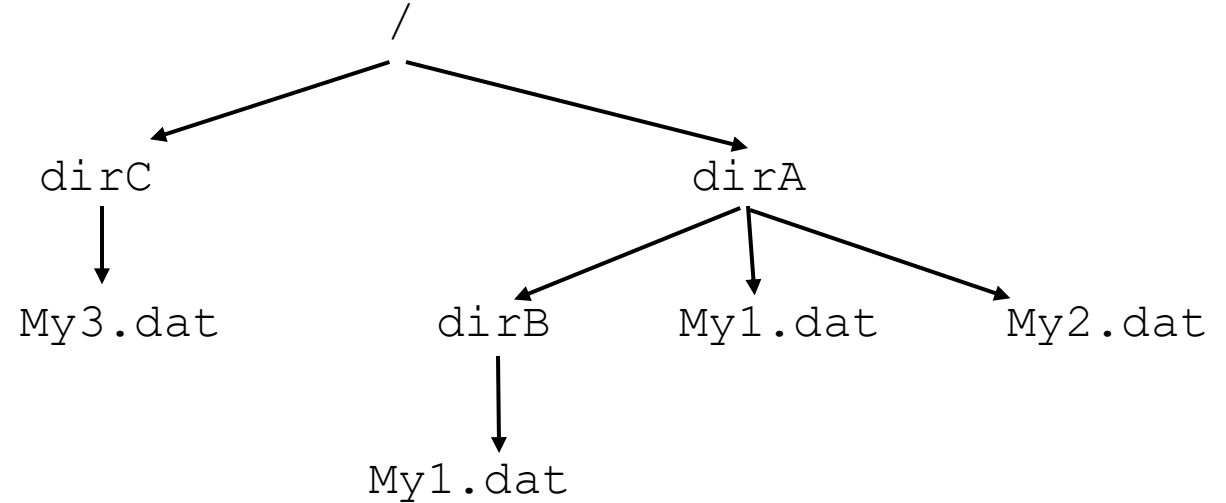
```
>cp /etc/passwd /tmp/garbage
```

```
>cp /etc/passwd /dev/tty
```

Demo Task1

# Hierarchical file organization

A Unix file system has a hierarchical tree structure where internal nodes are directories and leaf nodes are files

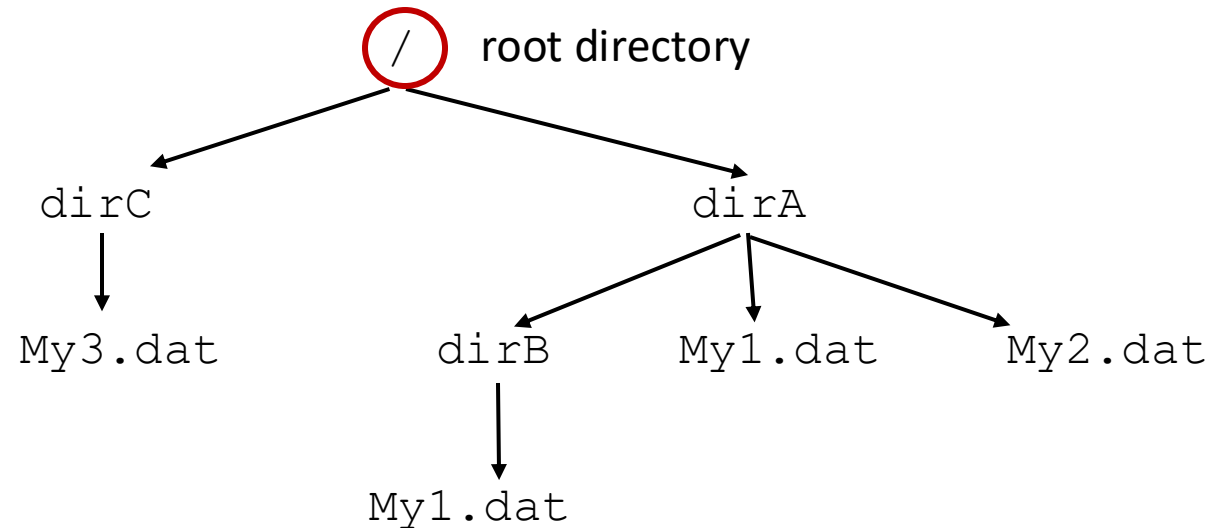


The absolute or **fully-qualified** pathname uniquely specifies a file, e.g. `/dirA/My1.dat` differs from `/dirA/dirB/My1.dat`.

We can also use a **relative** pathname, which begins from the current directory instead of the root directory, e.g., `../My2.dat` (if current working directory is `dirB`).

# Hierarchical file organization

A Unix file system has a hierarchical tree structure where internal nodes are directories and leaf nodes are files



The absolute or **fully-qualified** pathname uniquely specifies a file, e.g. `/dirA/My1.dat` differs from `/dirA/dirB/My1.dat`.

We can also use a **relative** pathname, which begins from the current directory instead of the root directory, e.g., `../My2.dat` (if current working directory is `dirB`).

# Current working directory

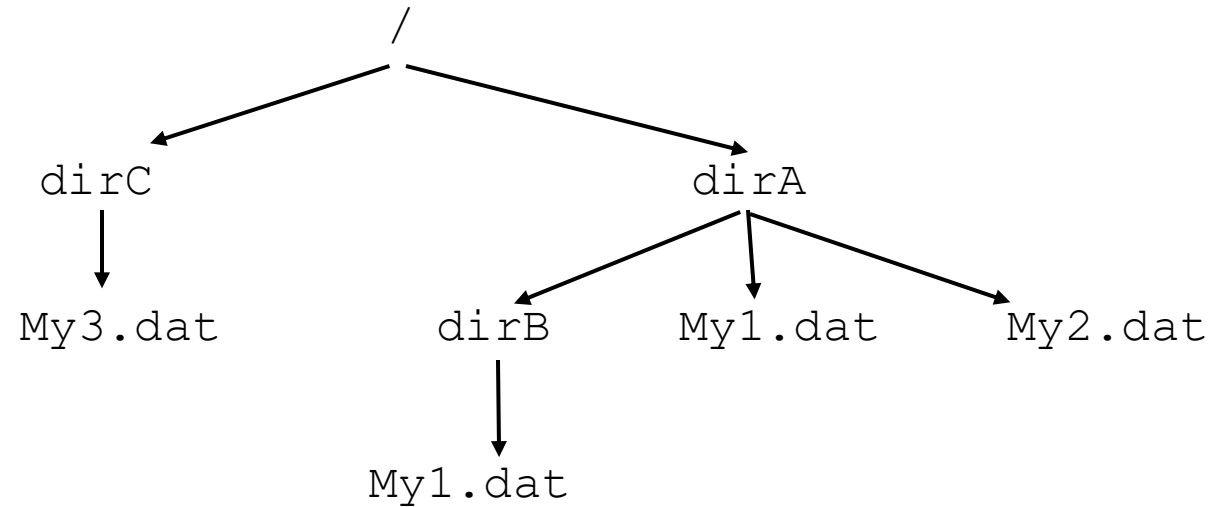
At anytime, every process has an associated directory called the current working directory (**cwd**)

Denoted by a dot ".", e.g., "mv ../file1 ."

The cwd associated with a user's login shell is called the user's **home** directory.

**pwd** prints the name of the cwd.

A relative pathname always starts with the path to the cwd.



# Current working directory

The C library function `getcwd` returns the pathname of the current working directory

```
char *getcwd(char *buf, size_t size)
```

`size` specifies maximum length pathname. If longer than the maximum, returns `NULL` and sets `errno` to `ERANGE`.

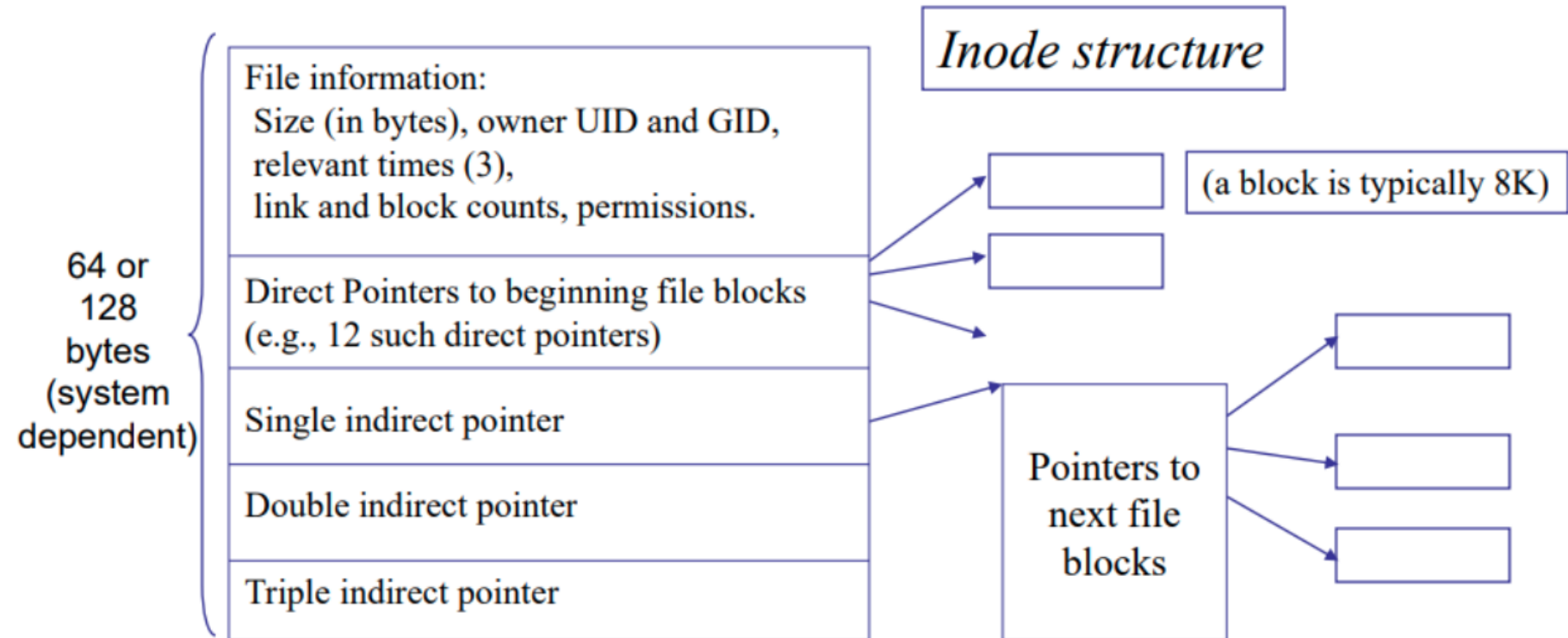
```
#include <unistd.h>
#include <stdio.h>
#include <errno.h>
int main() {
    char cwd[1024];
    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current working dir: %s\n", cwd);
    else perror("getcwd() error");
    return 0;
}
```

Demo Task2

# File representation

Information about a filesystem structure is stored both on disk and main memory.

Unix uses a logical structure called **i-node** to store the information about a file on disk – each file in a file system is represented by an i-node:

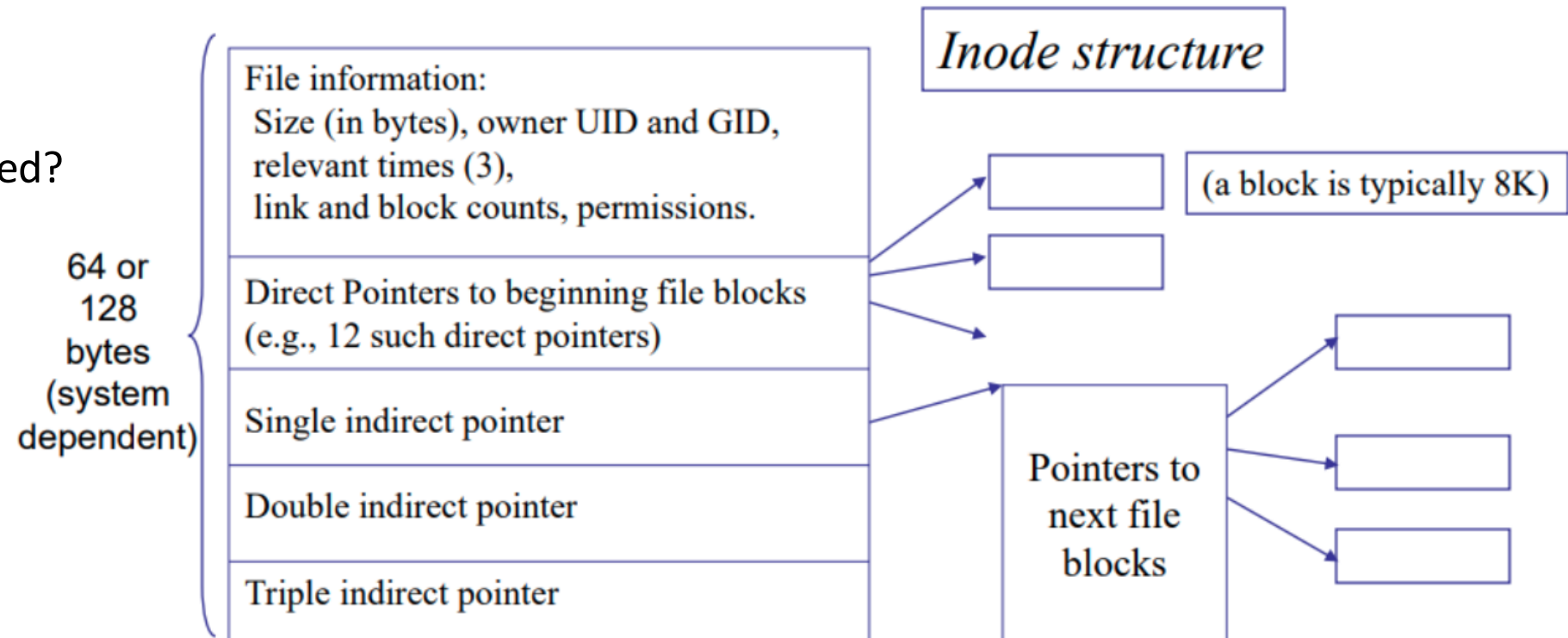


# File representation

Information about a filesystem structure is stored both on disk and main memory.

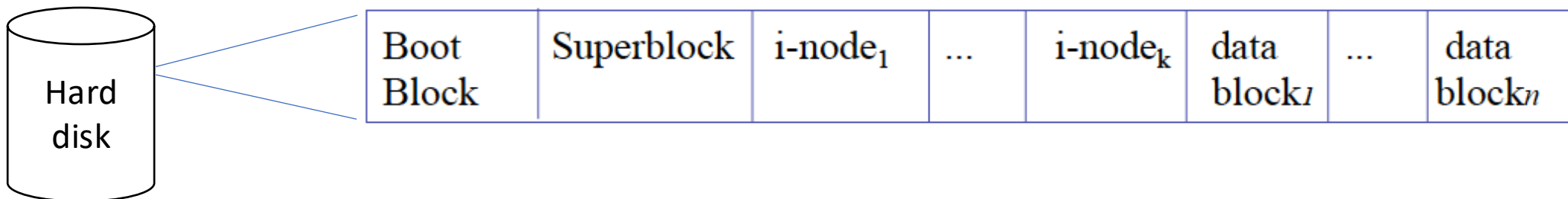
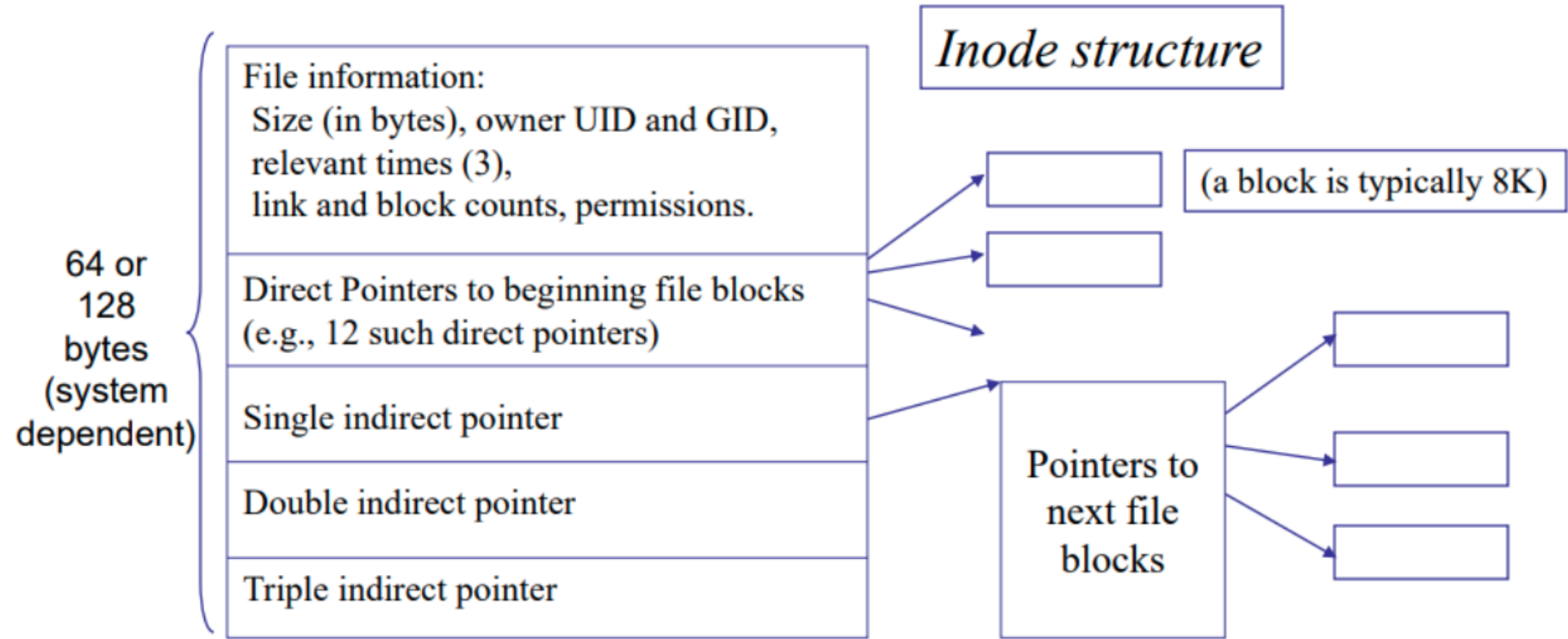
Unix uses a logical structure called **i-node** to store the information about a file on disk – each file in a file system is represented by an i-node:

Where is the file name stored?



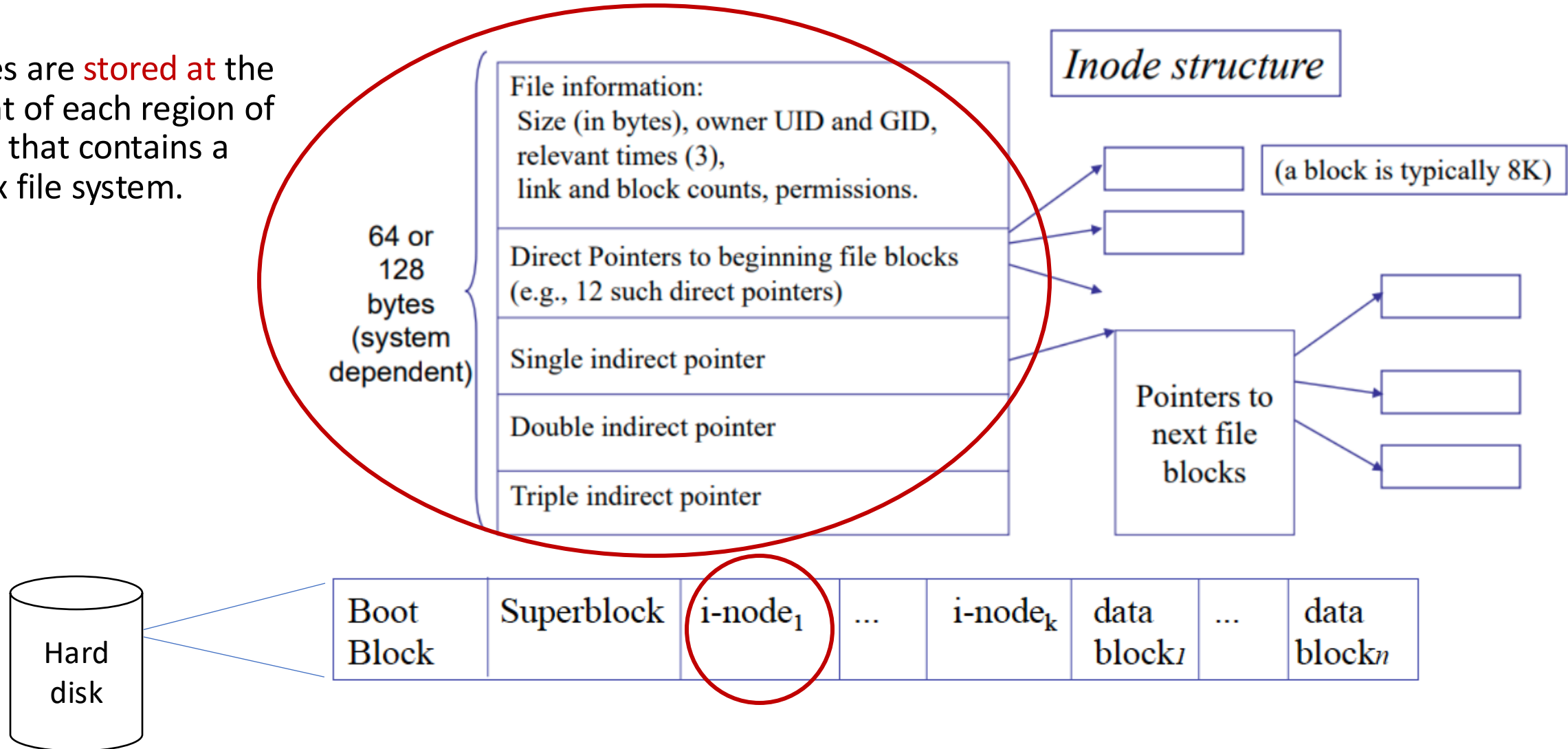
# Where is i-node stored? And how can we find the i-node of a file?

i-nodes are **stored at** the front of each region of **disk** that contains a Unix file system.



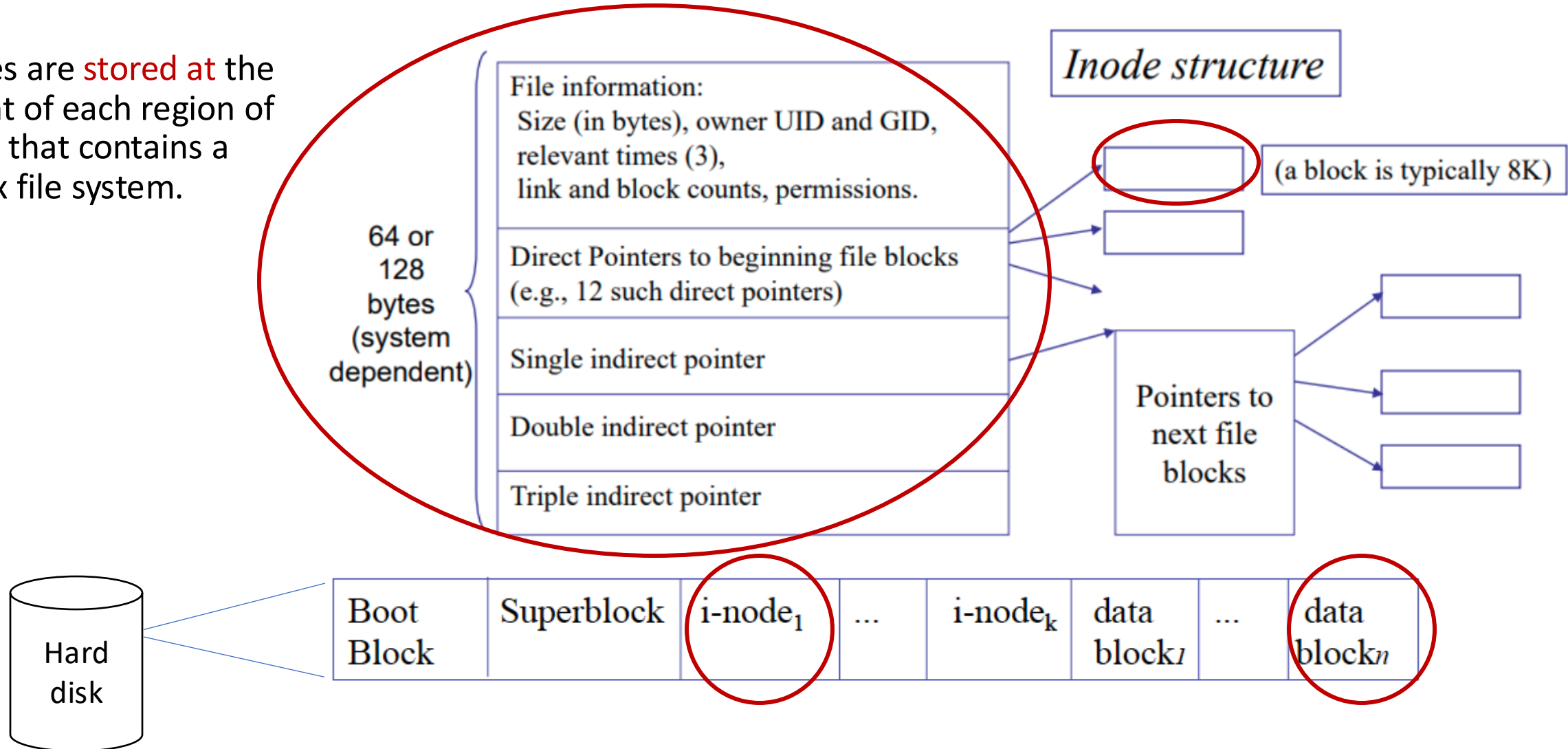
# Where is i-node stored? And how can we find the i-node of a file?

i-nodes are **stored at** the front of each region of **disk** that contains a Unix file system.



# Where is i-node stored? And how can we find the i-node of a file?

i-nodes are **stored at** the front of each region of **disk** that contains a Unix file system.

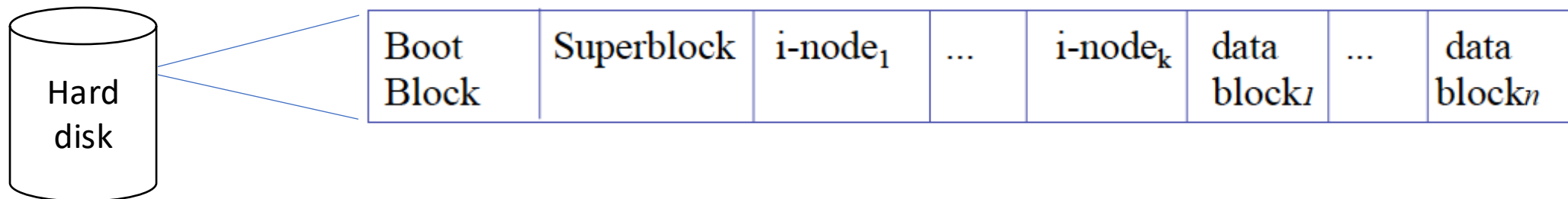


# Where is i-node stored? And how can we find the i-node of a file?

i-nodes are stored at the front of each region of disk that contains a Unix file system.

A regular file has a number called the **i-number**, which is an index into an array of the i-nodes on disk.

Each i-node corresponds to a unique i-number (index).

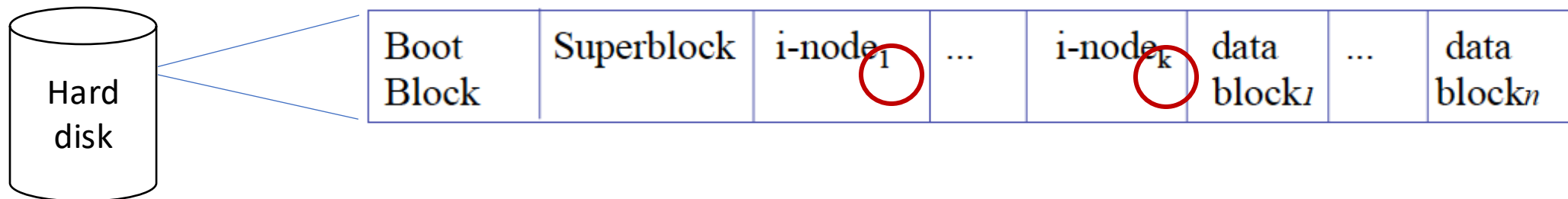


# Where is i-node stored? And how can we find the i-node of a file?

i-nodes are stored at the front of each region of disk that contains a Unix file system.

A regular file has a number called the **i-number**, which is an index into an array of the i-nodes on disk.

Each i-node corresponds to a unique i-number (index).



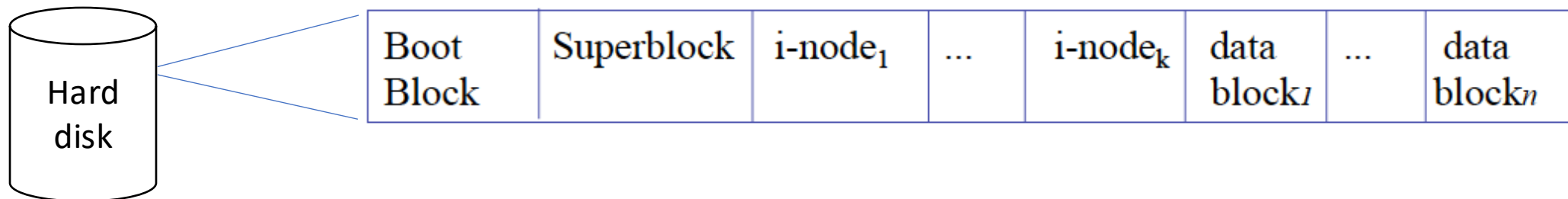
# Where is i-node stored? And how can we find the i-node of a file?

i-nodes are stored at the front of each region of disk that contains a Unix file system.

A regular file has a number called the **i-number**, which is an index into an array of the i-nodes on disk.

Each i-node corresponds to a unique i-number (index).

How can we find the i-number of a file, given the file name?



# Where is i-node stored? And how can we find the i-node of a file?

i-nodes are stored at the front of each region of disk that contains a Unix file system.

A regular file has a number called the **i-number**, which is an index into an array of the i-nodes on disk.

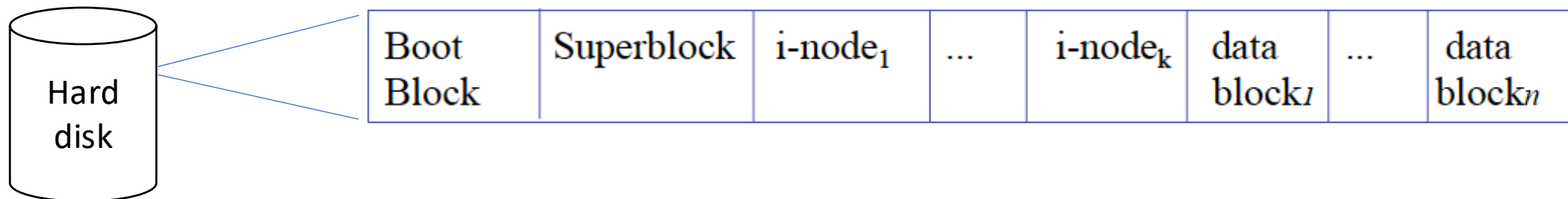
Each i-node corresponds to a unique i-number (index).

How can we find the i-number of a file, given the file name?

A directory contains a list of entries mapping file names to i-numbers (hence the i-node).

A <name, i-node> pair is called a **link**. You can create many links for a file (multiple names).

i-nodes are the hidden part, while directories are the visible structure of the Unix file system.



# Where is i-node stored? And how can we find the i-node of a file?

i-nodes are stored at the front of each region of disk that contains a Unix file system.

A regular file has a number called the **i-number**, which is an index into an array of the i-nodes on disk.

Each i-node corresponds to a unique i-number (index).

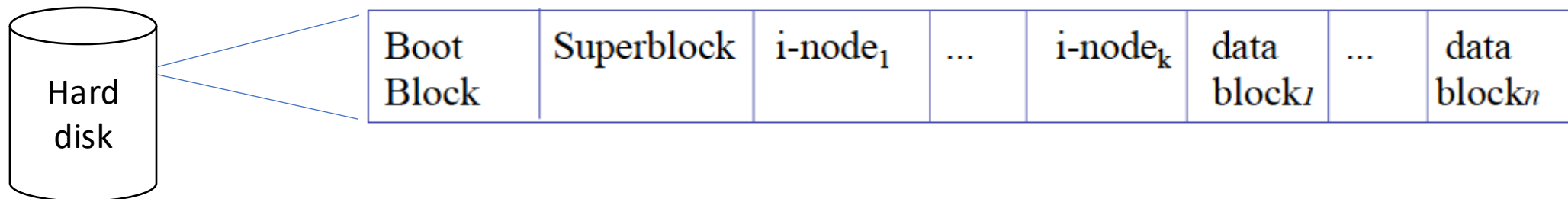
How can we find the i-number of a file, given the file name?

A directory contains a list of entries mapping file names to i-numbers (hence the i-node).

A <name, i-node> pair is called a **link**. You can create many links for a file (multiple names).

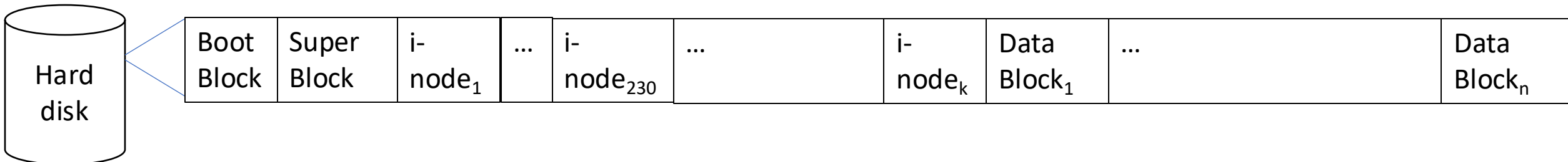
i-nodes are the hidden part, while directories are the visible structure of the Unix file system.

Hence, the correspondence between a file name and its i-node is stored in the directory file.



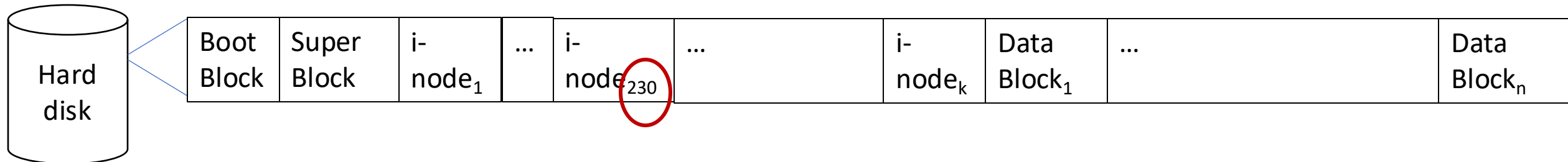
# An example directory file

| i-number | File name |
|----------|-----------|
| 123      | .         |
| 247      | ..        |
| 220      | dirB      |
| 230      | My1.dat   |



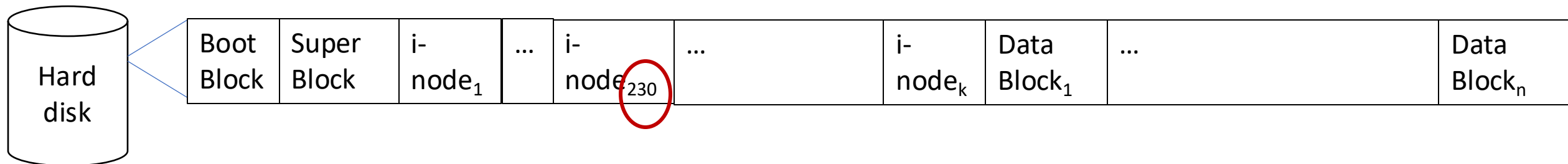
# An example directory file

| i-number | File name |
|----------|-----------|
| 123      | .         |
| 247      | ..        |
| 220      | dirB      |
| 230      | My1.dat   |



# An example directory file: hard link

| i-number | File name |
|----------|-----------|
| 123      | .         |
| 247      | ..        |
| 220      | dirB      |
| 230      | My1.dat   |
| 230      | My2.dat   |

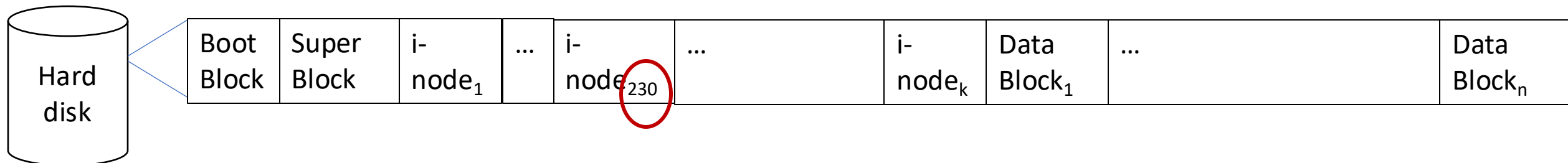


# An example directory file: hard link

Hard link creation command:

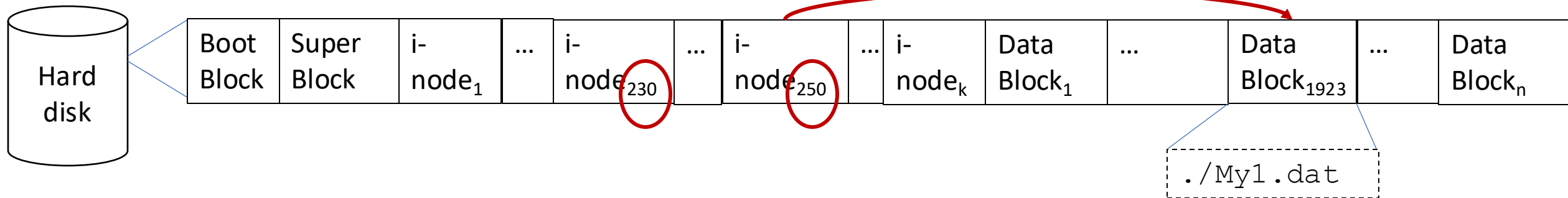
```
ln My1.dat My2.dat
```

| i-number | File name |
|----------|-----------|
| 123      | .         |
| 247      | ..        |
| 220      | dirB      |
| 230      | My1.dat   |
| 230      | My2.dat   |



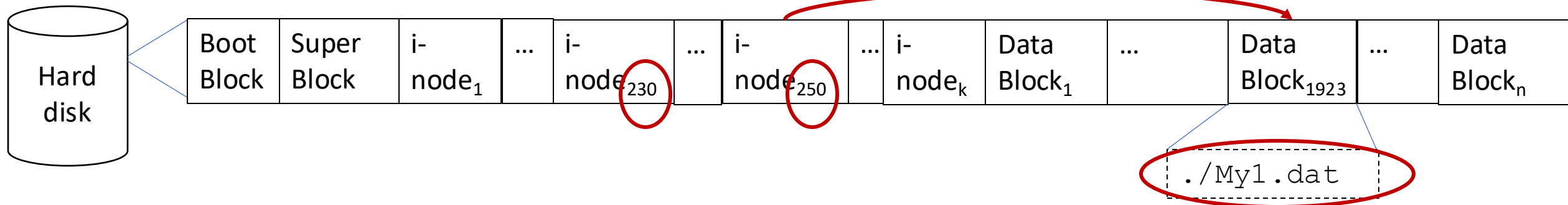
# An example directory file: symbolic link

| i-number | File name      |
|----------|----------------|
| 123      | .              |
| 247      | ..             |
| 220      | dirB           |
| 230      | My1.dat        |
| 230      | My2.dat        |
| 250      | My1SymLink.dat |



# An example directory file: symbolic link

| i-number | File name      |
|----------|----------------|
| 123      | .              |
| 247      | ..             |
| 220      | dirB           |
| 230      | My1.dat        |
| 230      | My2.dat        |
| 250      | My1SymLink.dat |

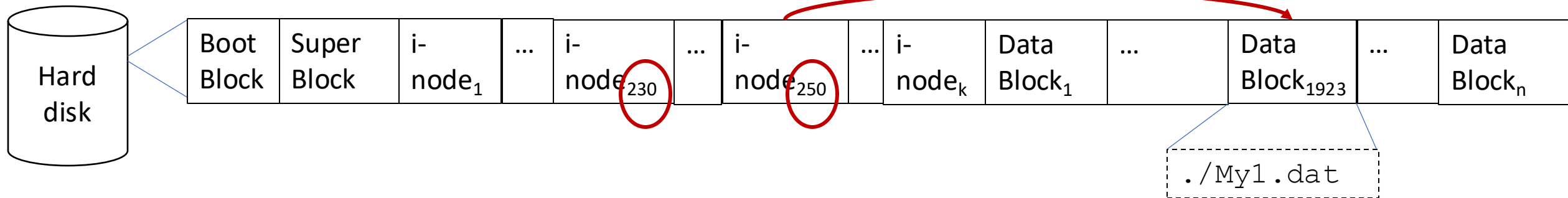


# An example directory file: symbolic link

Symbolic link creation command:

```
ln -s My1.dat My1SymLink.dat
```

| i-number | File name      |
|----------|----------------|
| 123      | .              |
| 247      | ..             |
| 220      | dirB           |
| 230      | My1.dat        |
| 230      | My2.dat        |
| 250      | My1SymLink.dat |



# Hard Link vs Symbolic Link

Hard links must be on the same file system, while symbolic links can cross file systems.

Hard linked files stay linked even if either one is moved (unless the move crosses file system boundaries, triggering copy-and-delete). Symbolic linked files break if the original is moved.

Hard linked files are co-equal (data is deleted only when all hard links are deleted), while the original is special in symbolic links (deleting the original deletes the data, deleting the symbolic link does not delete the data).

Symbolic link can point at any target, but most OS/filesystems disallow hardlinking directories to prevent cycles.

Symbolic link can require special support from file system walking tools.

-- from <https://superuser.com/questions/37910/what-are-the-advantages-of-symlinks-over-hard-links-and-viceversa>

# Where is i-node stored? And how can we find the i-node of a file?

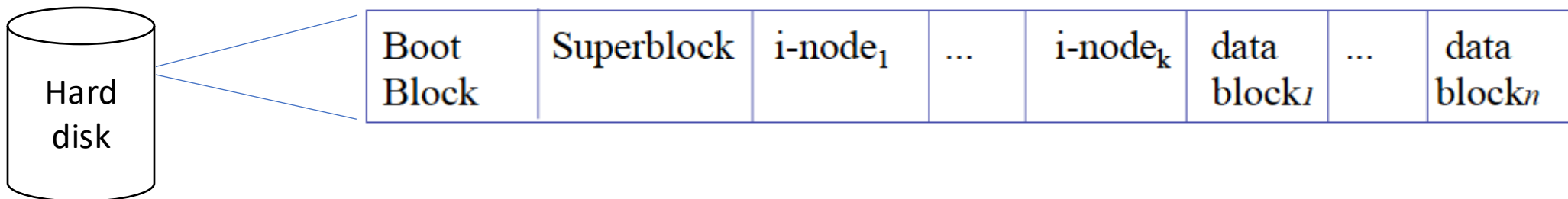
**Name resolution** is a function to convert a pathname to an i-node number.

When a program references a file by pathname, OS traverses the file system tree to find the filename and the i-number in the appropriate directory.

Once it has the i-number, the OS can determine other information about the file by accessing the i-node.

Usually, the OS keeps copies of the i-nodes for active files in memory for efficiency.

Remark: directories are disk resident files that can be read by any program. In many aspects, they are treated the same way as regular files.



# How large a file can be?\*

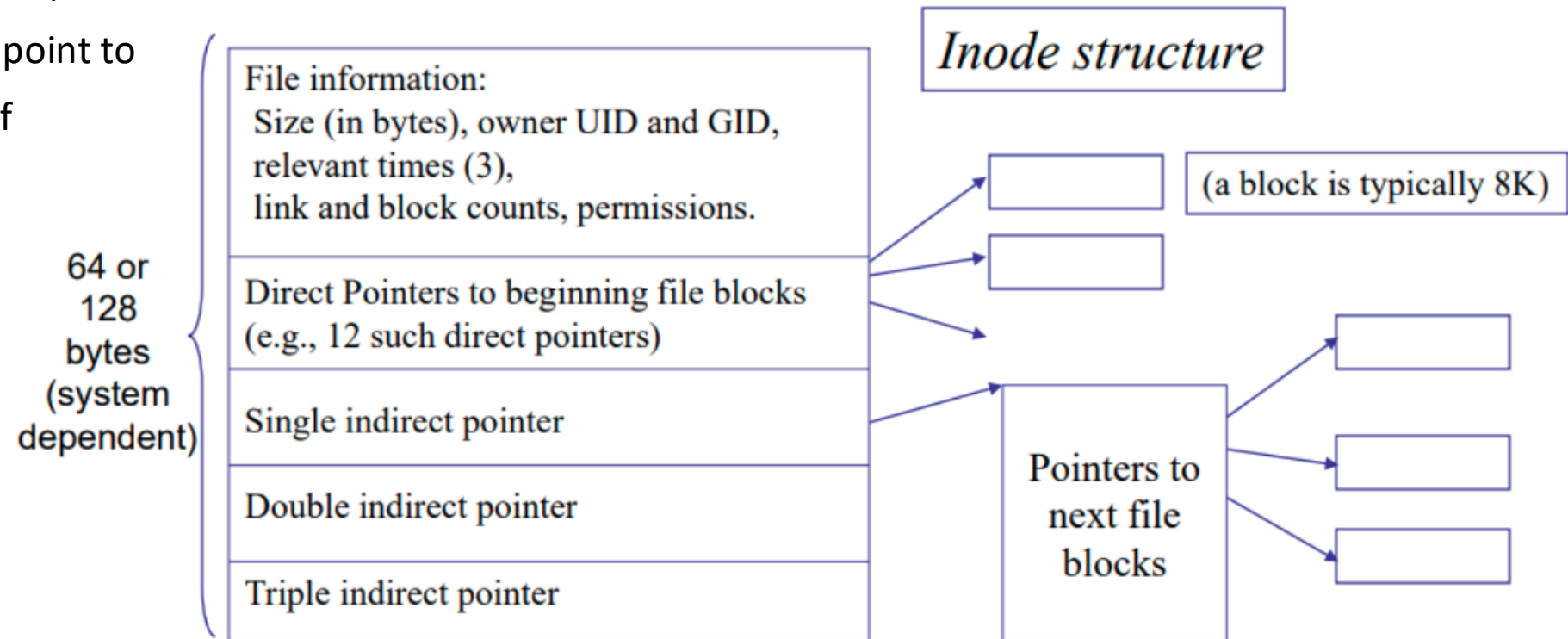
12 **direct pointer** can point to  $12 \times 8\text{KB} = 96\text{KB}$  of file content.

A **single indirect pointer** points to a block of direct pointers. A block can contain  $8\text{KB}/4\text{bytes} = 2\text{K}$  pointers = 2048 pointers. 2048 direct pointers can point to  $2048 \times 8\text{KB} = 16\text{MB}$  of file content.

A **double indirect pointer** points to 2048 single indirect pointers, that is  $2048 \times 16\text{MB} = 32\text{GB}$  of file content.

Similarly, a **triple indirect pointer** points to **64TB** of file content.

So one I-node can at the most point to  
 $64\text{TB} + 32\text{GB} + 16\text{MB} + 96\text{KB}$  of  
file content.



# How are files accessed in user programs?

Within C programs, a file is accessed either by **file descriptors** or by **file pointers**, which provide logical names (handles) for performing device-independent I/O.

The Unix file system calls use file descriptors (via `open`, `read`, `write`, `close`, `fsync`, `lseek`, and `ioctl`).

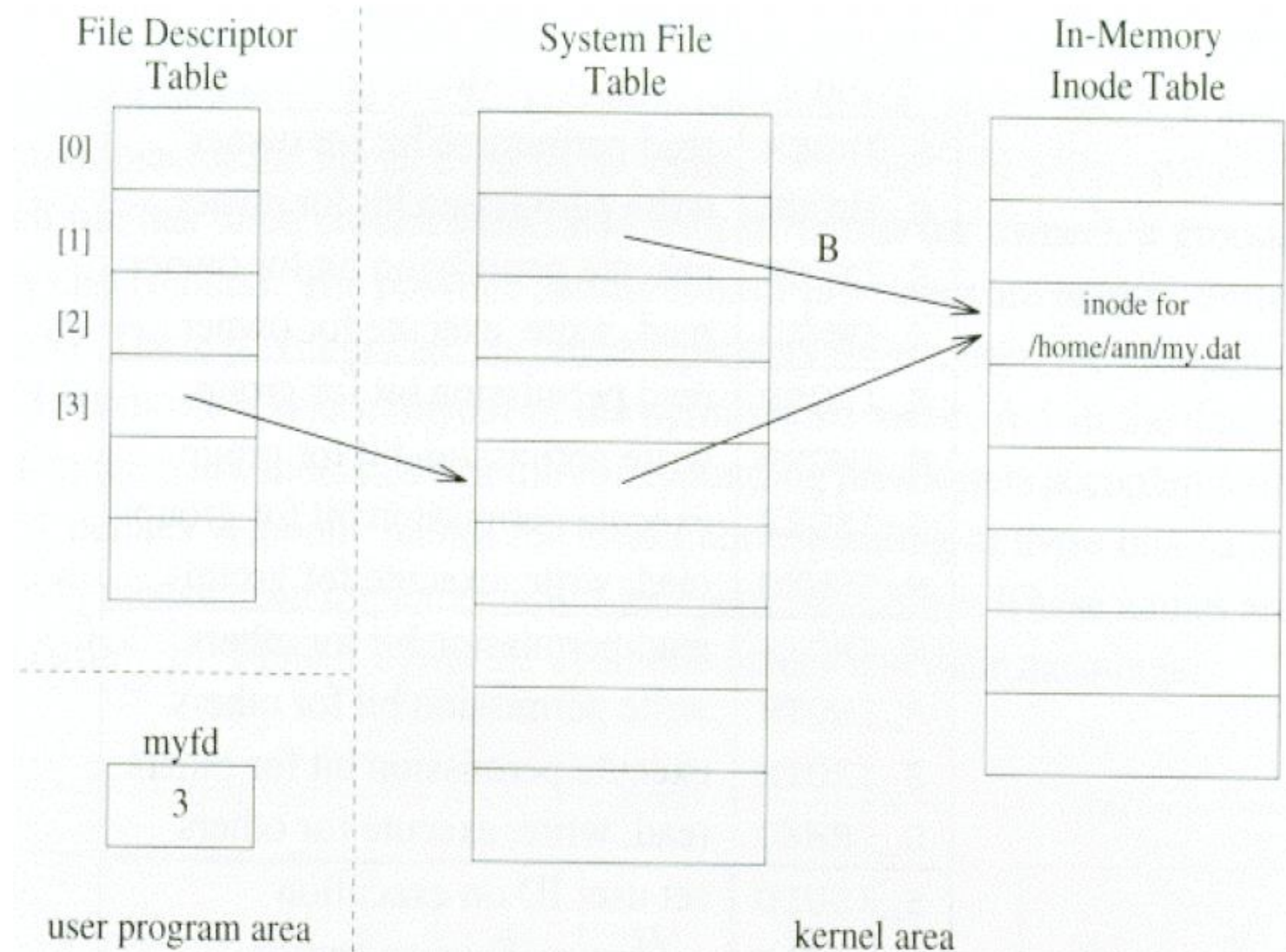
The ANSI C I/O library uses file pointers (via `fopen`, `fscanf`, `fprintf`, `feof`, `fflush`, `fseek`, `fread`, `fwrite`, `fclose`, etc.).

File descriptors (in `unistd.h`) for standard input, standard output, and standard error files are `STDIN_FILENO`, `STDOUT_FILENO`, and `STDERR_FILENO`; while file pointers (in `stdio.h`) are `stdin`, `stdout`, and `stderr`;

# File descriptors

SFT entry contains information about whether a file is opened for read/write, permission, lock, read/write offset etc.

Several entries in SFT may point to the same physical file.



# File descriptors

Three files are opened automatically:

`STDIN_FILENO`: standard input

`STDOUT_FILENO`: standard output

`STDERR_FILENO`: standard error

Corresponding to constants 0, 1, 2 in `unistd.h`

When new files are opened, it is assigned the lowest available FD.

Accessing files for I/O is a three-step process, whether it is a regular file or a device:

Open the file for I/O

Read and write to the file

Close the file when finished with I/O

# File descriptor: open a file

```
int open(const char* pathname, int flags)
int open(const char* pathname, int flags, mode_t mode)
```

## Open a file

`pathname` : absolute or relative path to the file

`flags` : must include one of the following access modes: `O_RDONLY`, `O_WRONLY`, `O_RDWR`, and bitwise-or'd (i.e. using the `|` operator) with zero or more of the following:

- `O_CREAT`: if the `pathname` does not exist, create the file as a regular file; must add the access permission mode parameter (e.g. `0644`).
- `O_TRUNC`: if the file already exists and is a regular file and the access mode allows writing, truncate the file to size 0.
- `O_APPEND`: the file is opened in append mode.

...

Returns the opened file's file descriptor or `-1` if an error occurred (the `errno` is set accordingly)

```
int fd = open("someFile", O_RDONLY);
```

# File descriptor: read a file

```
bytes = read(fd, buffer, count)
```

Read from file associated with `fd`; place `count` bytes into `buffer`

`fd` : file descriptor to read from

`buffer` : pointer to an array

`count` : number of bytes to read

Returns number of bytes read or `-1` if an error occurred

```
int fd = open("someFile", O_RDONLY);
```

```
char buffer[4];
```

```
int bytes = read(fd, buffer, 4*sizeof(char));
```

Demo Task3

# File descriptor: write a file

```
bytes = write(fd, buffer, count)
```

Write contents of `buffer` to the file associated with `fd`; write `count` bytes into the file

`fd` : file descriptor to write to

`buffer` : pointer to an array

`count` : number of bytes to write

Returns number of bytes written or `-1` if an error occurred

```
int fd = open("someFile", O_WRONLY|O_CREAT|O_TRUNC, 0644);
```

```
char buffer[4];
```

```
int bytes = write(fd, buffer, 4*sizeof(char));
```

```
fsync(fd);
```

Demo Task4

# File descriptor: close a file

```
return_val = close(fd)
```

Closes an open file descriptor

Returns 0 on success, -1 on error

# File pointers

A file pointer points to a data structure `FILE`, called a **file structure** in the user area of the process.

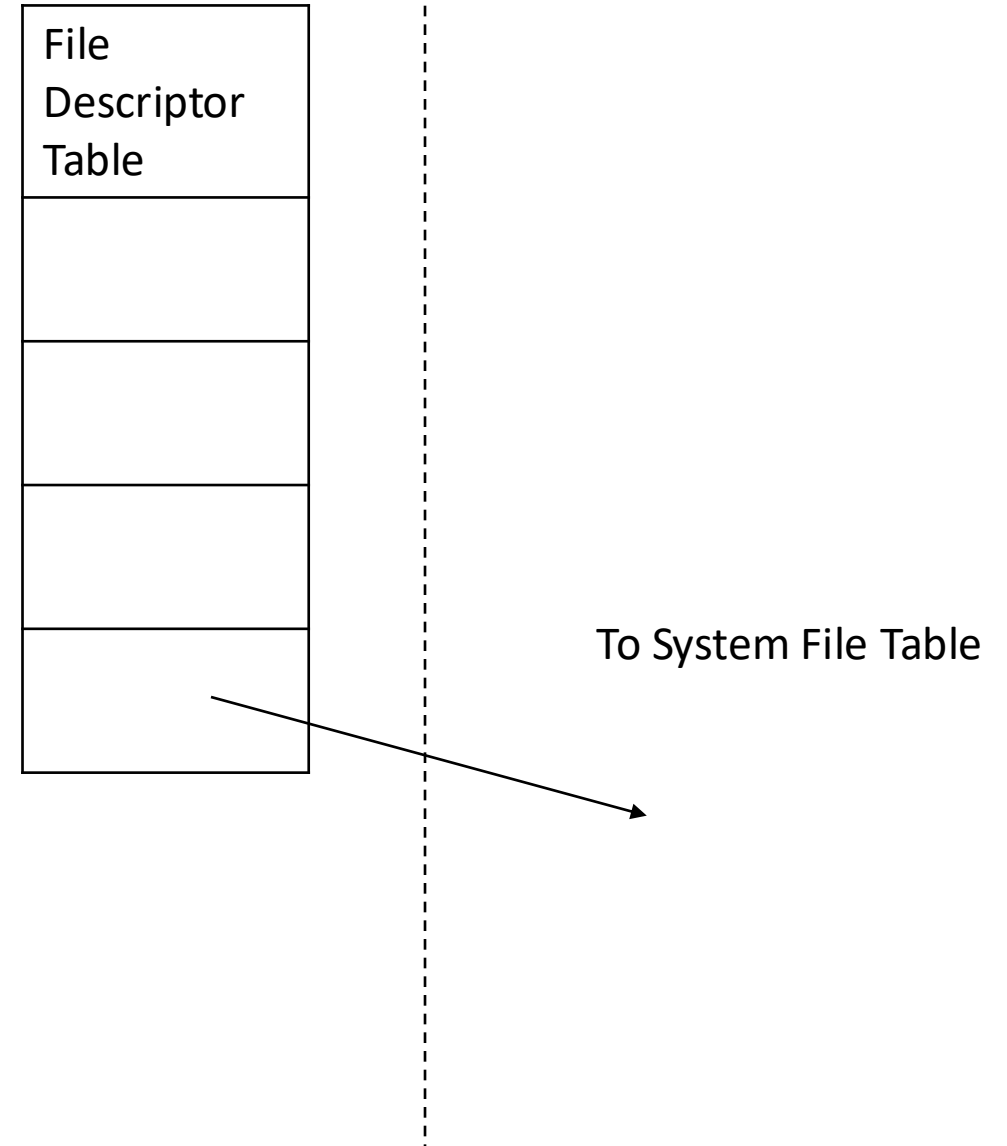
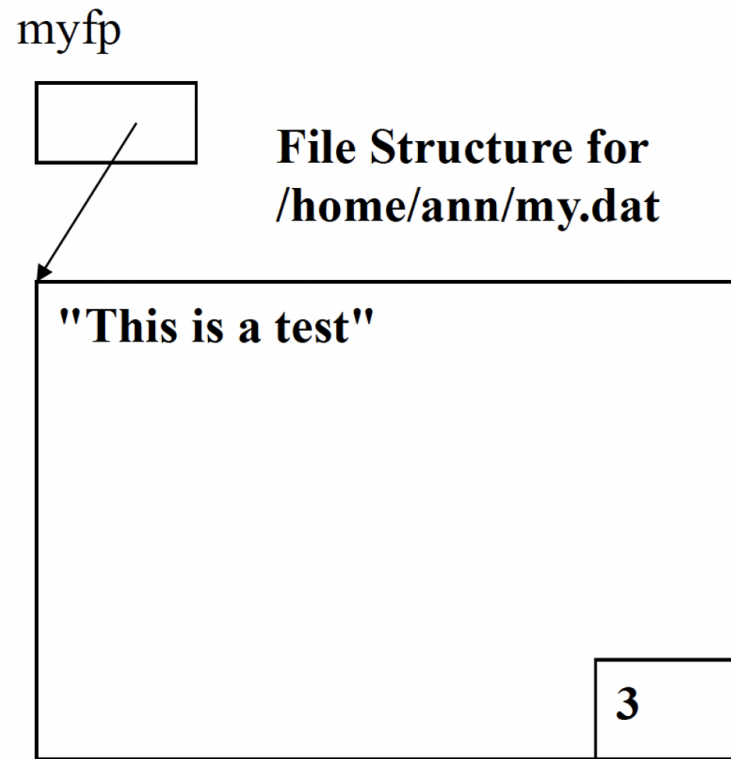
A file structure **contains** a **buffer** and a **file descriptor** (so a file pointer is a handle to a handle)

The following code segment open the file `/home/ann/my.dat` for output and then writes a string to the file.

```
#include <stdio.h>
FILE *myfp;
if ((myfp = fopen("/home/ann/my.dat", "w")) == NULL)
    fprintf(stderr, "Could not fopen file\n");
else
    fprintf(myfp, "This is a test");
```

Demo Task5

# File pointers



# File pointer and FILE

File pointers are used (via the `FILE` data type) in the following higher-level I/O functions in C libraries:

`fopen()`

`printf()`

`scanf()`

`fflush()`

`feof()`

`fclose()`

# `fopen()` and `fclose()`

```
FILE *file_stream = fopen(path, mode)
```

Path: `char*`, absolute or relative path

Mode

`r` – open file for reading

`r+` - open file for reading and writing

`w` – overwrite file or create file for writing

`w+` - open for reading and writing; overwrites file

`a` – open file for appending (writing at end of file)

`a+` - open file for appending and reading

```
fclose(file_stream)
```

Closes the opened file represented by `file_stream`

# printf()

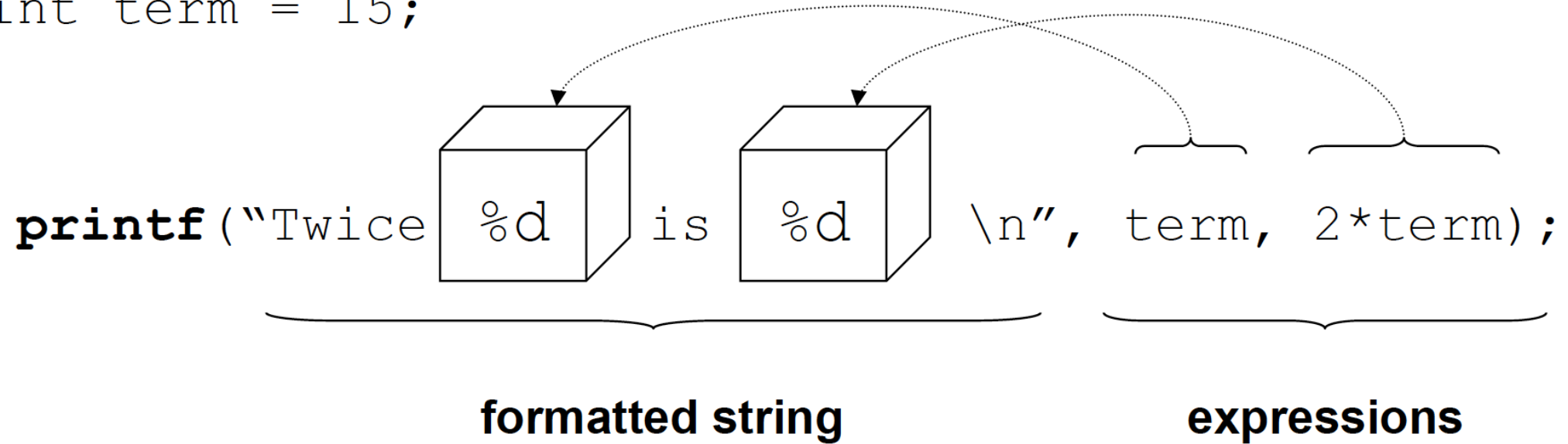
```
printf(formatted_string, ...)
```

`formatted_string`: string that describes the output information,  
variable types are escaped with %

The formatted string is followed by as many expressions as are  
referenced in the formatted string

# printf()

```
int term = 15;
```



# Escaping variable types

`%d,%i` – decimal integer

`%u` – unsigned decimal integer

`%o` – unsigned octal integer

`%x,%X` – unsigned hexadecimal integer

`%c` – character

`%s` – string or character array

`%f` – float

`%e,%E` – double (scientific notation)

`%g,%G` – double or float

`%%` – outputs a `%` character

## Demo Task6

### printf ( ) examples

```
printf("The sum of %d, %d, and %d is %d\n", 65, 87,  
33, 65+87+33);
```

**Output:** The sum of 65, 87, and 33 is 185

```
printf("Error %s occurred at line %d \n", emsg,  
lno);
```

**Output:** Error invalid variable occurred at line 27

```
printf("Hexadecimal form of %d is %x \n", 59, 59);
```

**Output:** Hexadecimal form of 59 is 3B

## scanf ( ) examples

```
scanf (formatted_string, ...)
```

Similar syntax as `printf`, only the formatted string represents the data that you are reading in.

Must pass variables by reference

Example:

```
scanf (" %d %c %s", &int_var, &char_var,  
string_var);
```

## scanf ( ) examples

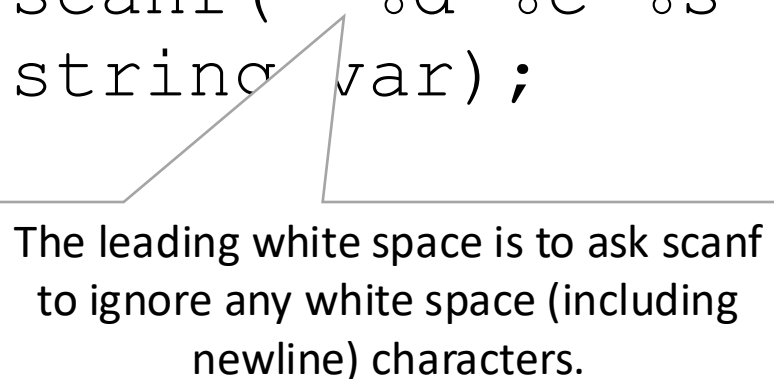
```
scanf (formatted_string, ...)
```

Similar syntax as `printf`, only the formatted string represents the data that you are reading in.

Must pass variables by reference

Example:

```
scanf (" %d %c %s", &int_var, &char_var,  
string_var);
```



The leading white space is to ask scanf to ignore any white space (including newline) characters.

## scanf () examples

```
scanf (formatted_string, ...)
```

Similar syntax as `printf`, only the formatted string represents the data that you are reading in.

Must pass variables by reference

Example:

```
scanf ("%d %c %s", &int_var, &char_var,  
string_var);
```

Other alternatives to flush the `'\n'` in input buffer include using `getchar ()` after a `scanf ()` call, using `'%*c'`, but the best is to use `fgets ()` to get a line instead of using `scanf`

# printf() and scanf() families

`fprintf(file_stream, formatted_string, ...)`

Prints to a file stream instead of stdout

`sprintf(char_array, formatted_string, ...)`

Prints to a character array instead of stdout

`int fprintf(FILE *stream, const char *format[, argument ]...);`

`int sprintf(char *buffer, const char *format[, argument]...);`

`fscanf(file_stream, formatted_string, ...)`

Reads from a file stream instead of stdin

`sscanf(char_array, formatted_string, ...)`

Reads from a string instead of stdin

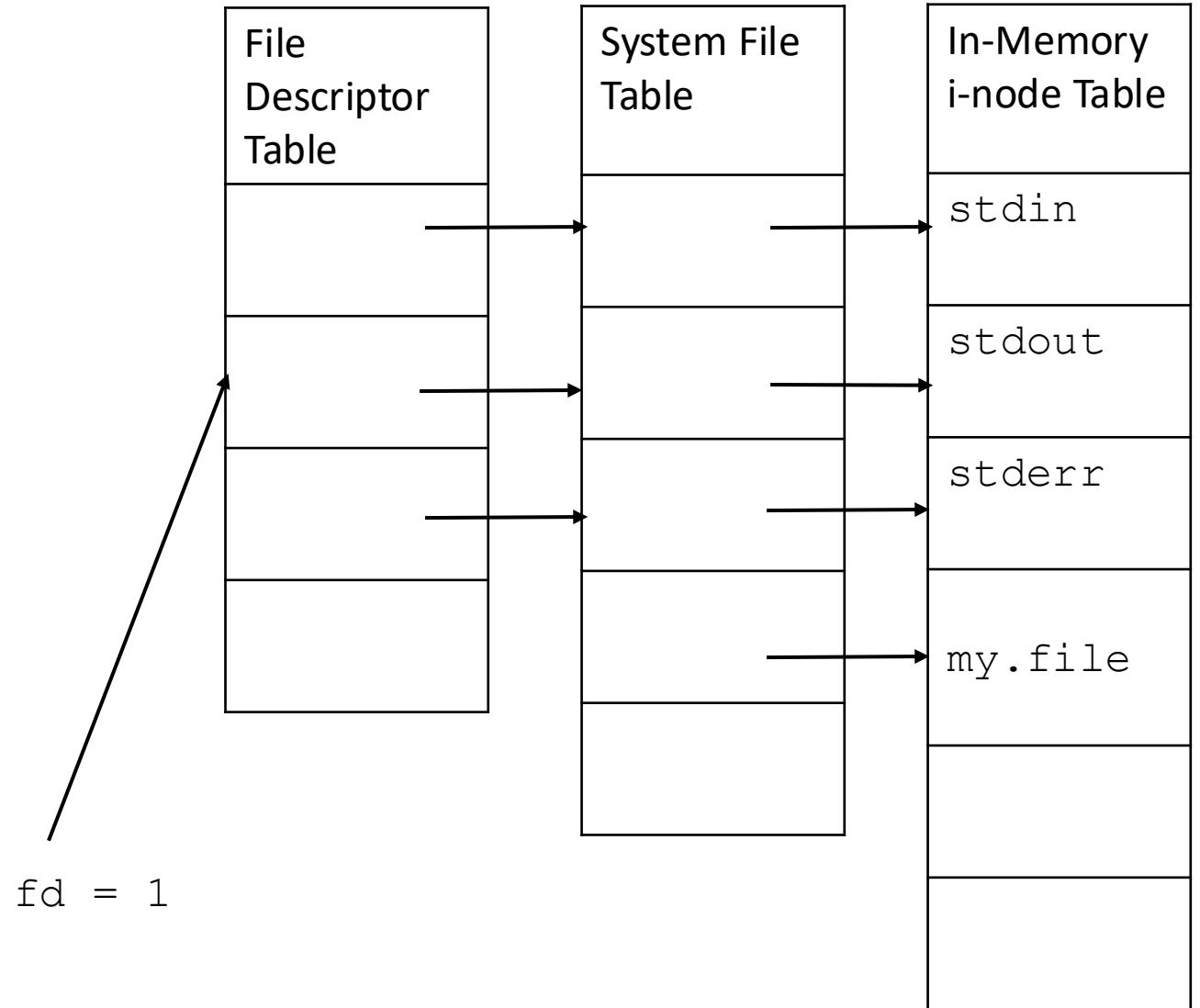
# I/O redirection

Recall: to access a file, a process uses a file descriptor, which is an index into the process file descriptor table, which in turn points to an entry in the system file table.

Redirection means that the process modifies its file descriptor table entry so that it points to a different entry in the system file table.

Consider the command `cat`, which reads from a file and outputs to `stdout`. The following command redirects `stdout` to `my.file`:

```
cat test > my.file
```



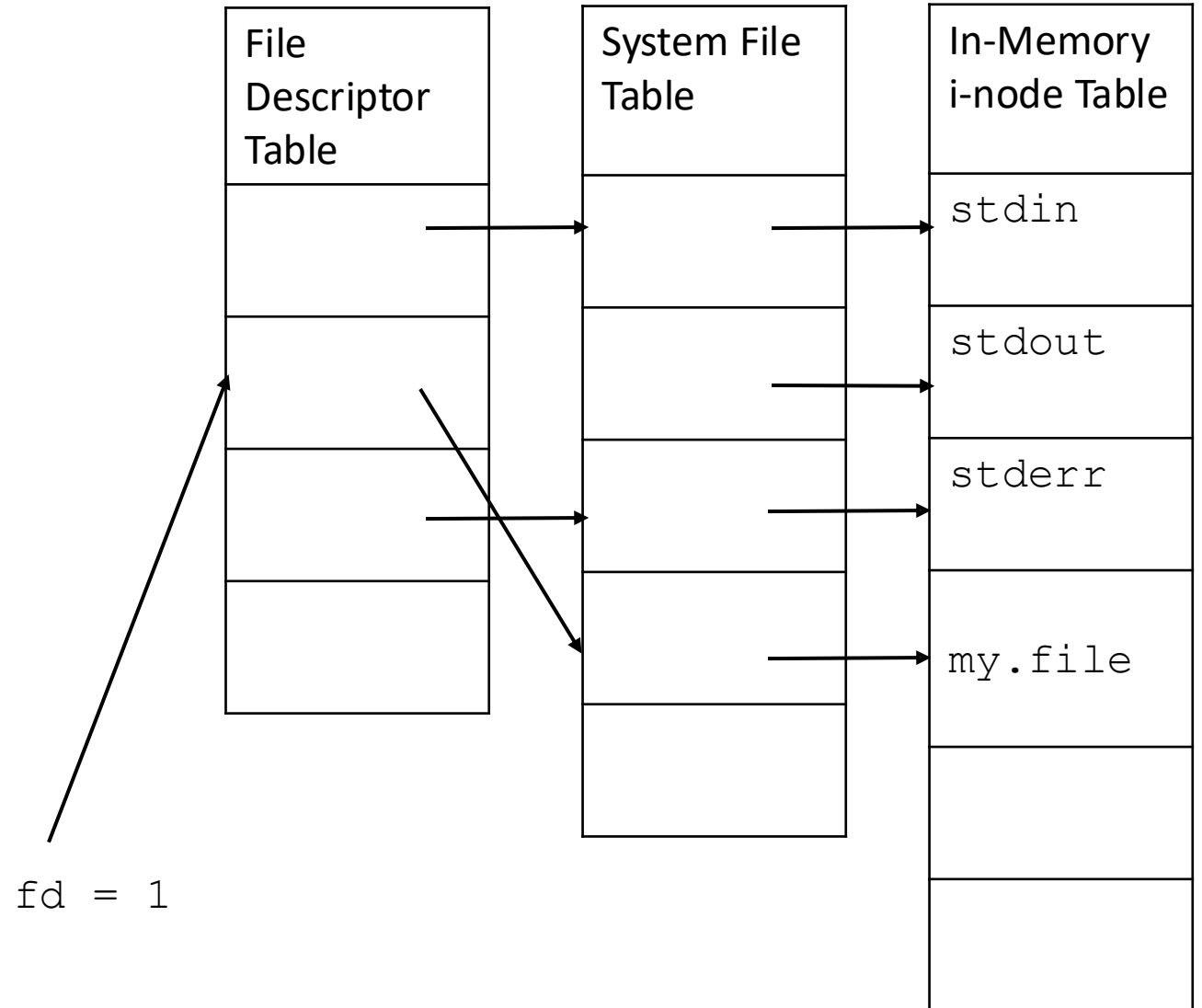
# I/O redirection

Recall: to access a file, a process uses a file descriptor, which is an index into the process file descriptor table, which in turn points to an entry in the system file table.

Redirection means that the process modifies its file descriptor table entry so that it points to a different entry in the system file table.

Consider the command `cat`, which reads from a file and outputs to `stdout`. The following command redirects `stdout` to `my.file`:

```
cat test > my.file
```



# I/O redirection realization using dup ( )

`int dup(int oldfd)` duplicates the given file descriptor to the lowest numbered unused file descriptor in the file descriptor table.

```
#include<stdio.h>
```

```
#include<fcntl.h>
```

```
#include<unistd.h>
```

```
char* cmd[] = {"/bin/ls", "-l", 0};
```

```
int main(int argc, char* argv[]){
```

```
    int fd = open(argv[1], O_WRONLY | O_CREAT, 0600);
```

```
    //fd will be 3; a file will be opened in write mode
```

```
    int fd2 = dup(fd); //duplicate the fd-th pointer to entry 4, the lowest available entry
```

```
    close(STDOUT_FILENO);
```

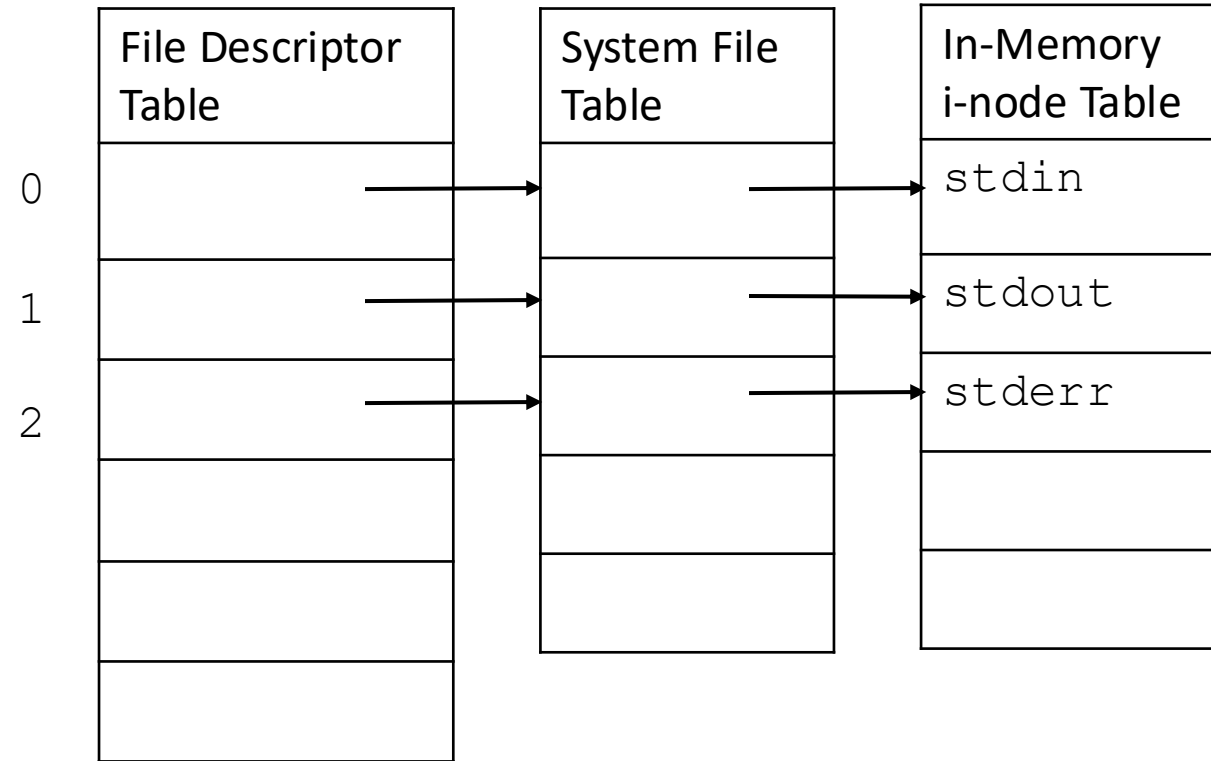
```
    dup(fd); //duplicate the fd-th pointer into entry 1
```

```
    execvp(cmd[0], cmd); //the old process image is replaced by the new process image for ls
```

```
    close(fd); //close file descriptor 3 in the parent process.
```

```
    return;
```

```
}
```



## Demo Task8

# I/O redirection realization using dup ( )

`int dup(int oldfd)` duplicates the given file descriptor to the lowest numbered unused file descriptor in the file descriptor table.

```
#include<stdio.h>
```

```
#include<fcntl.h>
```

```
#include<unistd.h>
```

```
char* cmd[] = {"/bin/ls", "-l", 0};
```

```
int main(int argc, char* argv[]){
```

```
    int fd = open(argv[1], O_WRONLY | O_CREAT, 0600);
```

```
    //fd will be 3; a file will be opened in write mode
```

```
    int fd2 = dup(fd); //duplicate the fd-th pointer to entry 4, the lowest available entry
```

```
    close(STDOUT_FILENO);
```

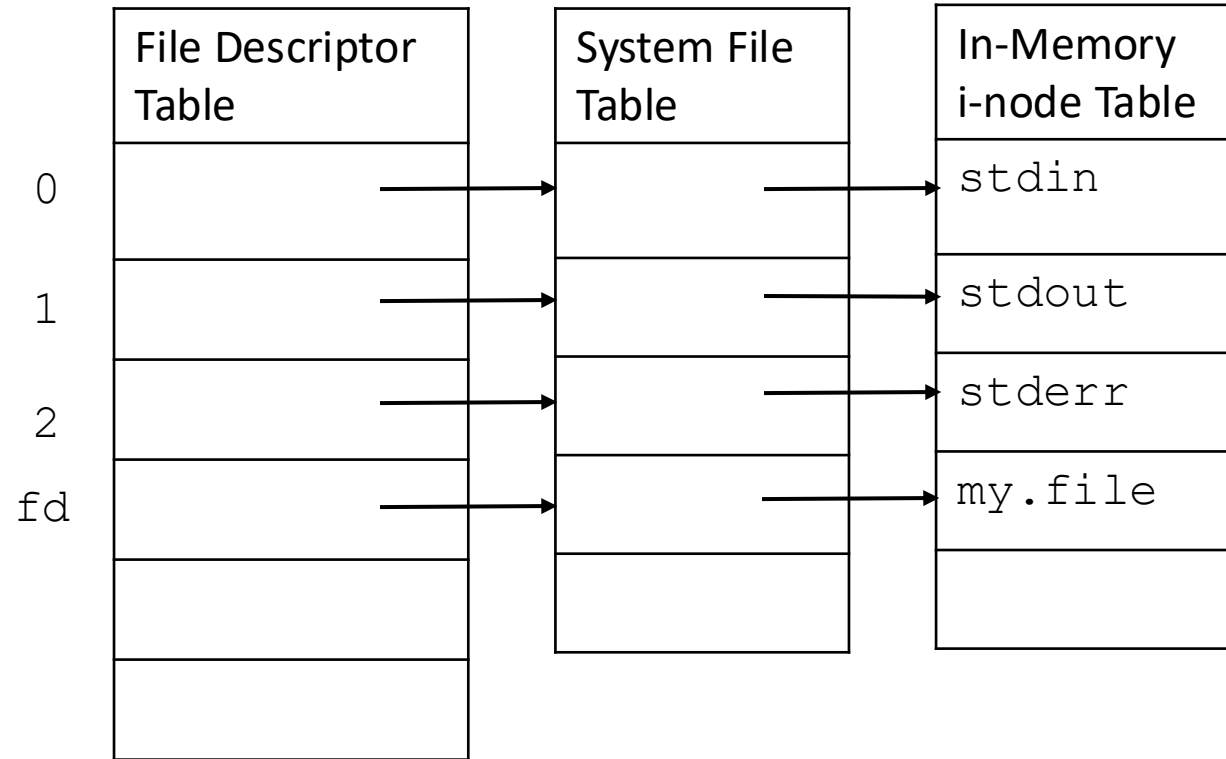
```
    dup(fd); //duplicate the fd-th pointer into entry 1
```

```
    execvp(cmd[0], cmd); //the old process image is replaced by the new process image for ls
```

```
    close(fd); //close file descriptor 3 in the parent process.
```

```
    return;
```

```
}
```



## Demo Task8

# I/O redirection realization using dup ( )

`int dup(int oldfd)` duplicates the given file descriptor to the lowest numbered unused file descriptor in the file descriptor table.

```
#include<stdio.h>
```

```
#include<fcntl.h>
```

```
#include<unistd.h>
```

```
char* cmd[] = {"/bin/ls", "-l", 0};
```

```
int main(int argc, char* argv[]){
```

```
    int fd = open(argv[1], O_WRONLY | O_CREAT, 0600);
```

```
    //fd will be 3; a file will be opened in write mode
```

```
    int fd2 = dup(fd); //duplicate the fd-th pointer to entry 4, the lowest available entry
```

```
    close(STDOUT_FILENO);
```

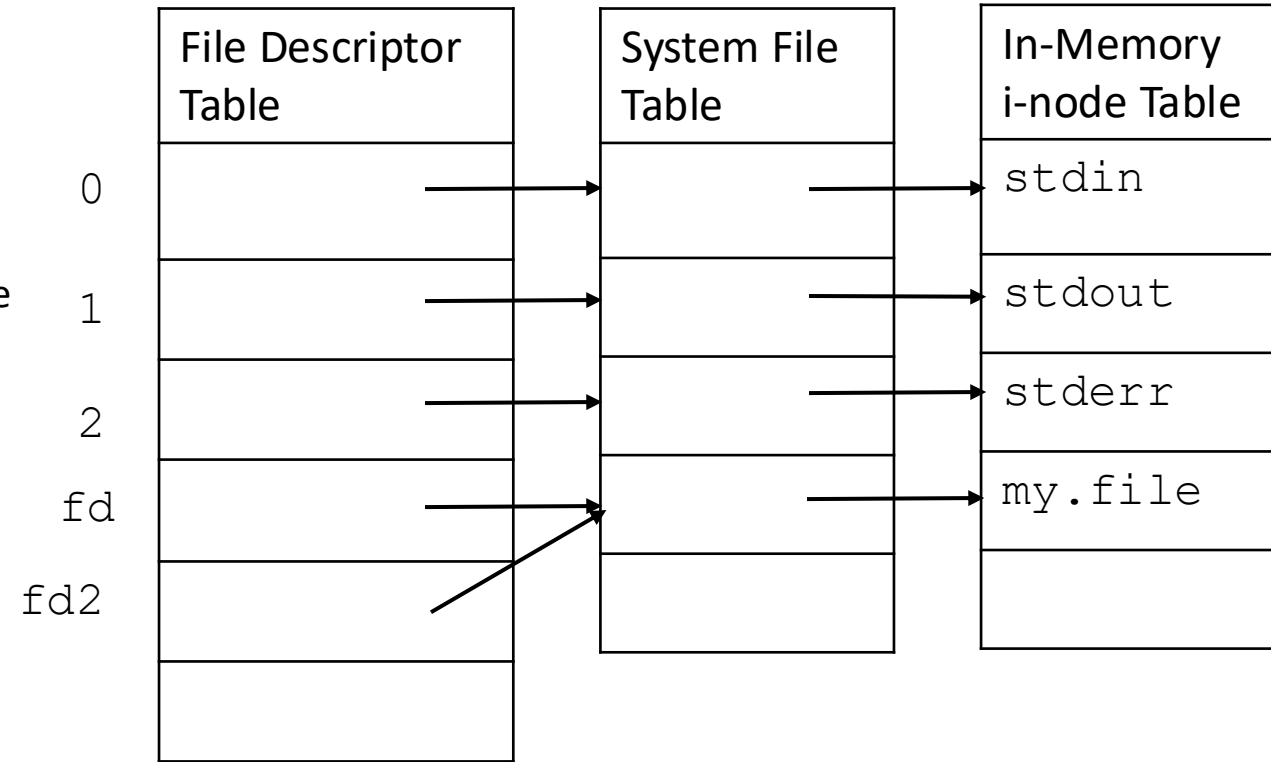
```
    dup(fd); //duplicate the fd-th pointer into entry 1
```

```
    execvp(cmd[0], cmd); //the old process image is replaced by the new process image for ls
```

```
    close(fd); //close file descriptor 3 in the parent process.
```

```
    return;
```

```
}
```



## Demo Task8

# I/O redirection realization using dup ( )

`int dup(int oldfd)` duplicates the given file descriptor to the lowest numbered unused file descriptor in the file descriptor table.

```
#include<stdio.h>
```

```
#include<fcntl.h>
```

```
#include<unistd.h>
```

```
char* cmd[] = {"/bin/ls", "-l", 0};
```

```
int main(int argc, char* argv[]){
```

```
    int fd = open(argv[1], O_WRONLY | O_CREAT, 0600);
```

```
    //fd will be 3; a file will be opened in write mode
```

```
    int fd2 = dup(fd); //duplicate the fd-th pointer to entry 4, the lowest available entry
```

```
    close(STDOUT_FILENO);
```

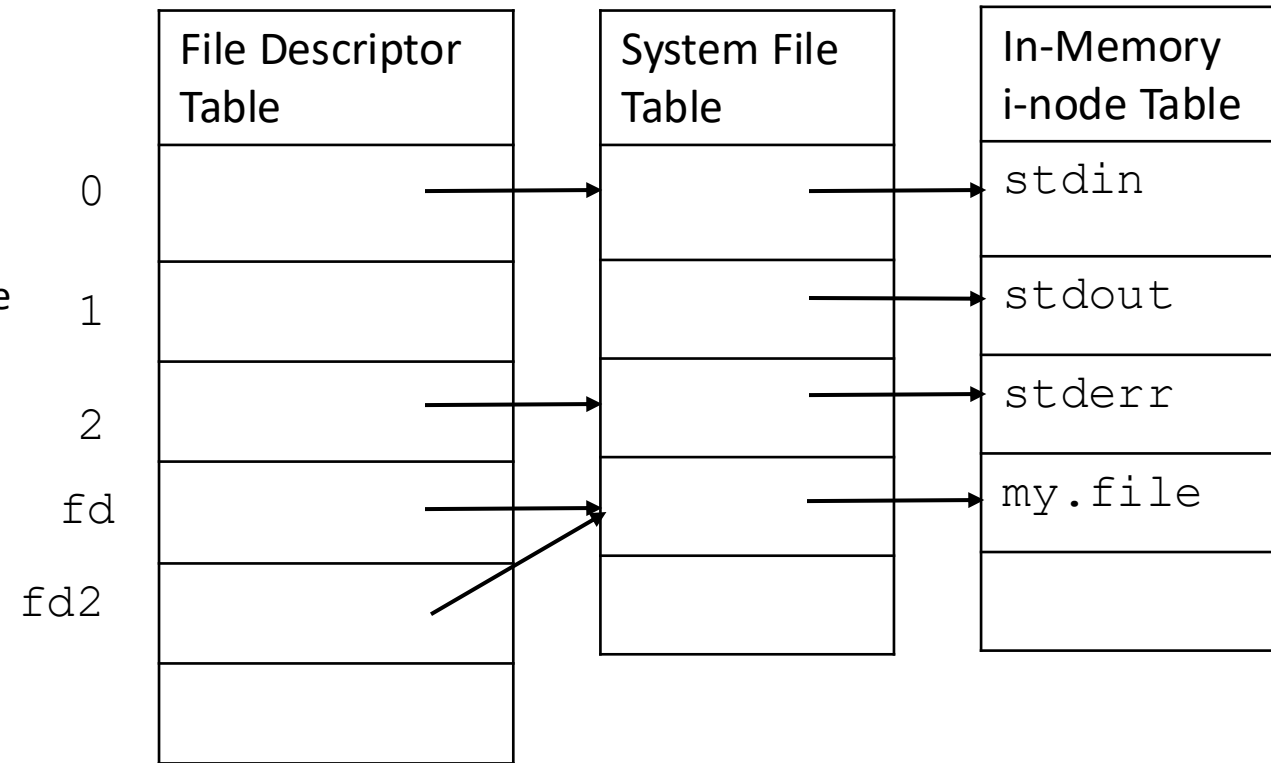
```
    dup(fd); //duplicate the fd-th pointer into entry 1
```

```
    execvp(cmd[0], cmd); //the old process image is replaced by the new process image for ls
```

```
    close(fd); //close file descriptor 3 in the parent process.
```

```
    return;
```

```
}
```



## Demo Task8

# I/O redirection realization using dup ( )

`int dup(int oldfd)` duplicates the given file descriptor to the lowest numbered unused file descriptor in the file descriptor table.

```
#include<stdio.h>
```

```
#include<fcntl.h>
```

```
#include<unistd.h>
```

```
char* cmd[] = {"/bin/ls", "-l", 0};
```

```
int main(int argc, char* argv[]){
```

```
    int fd = open(argv[1], O_WRONLY | O_CREAT, 0600);
```

```
    //fd will be 3; a file will be opened in write mode
```

```
    int fd2 = dup(fd); //duplicate the fd-th pointer to entry 4, the lowest available entry
```

```
    close(STDOUT_FILENO);
```

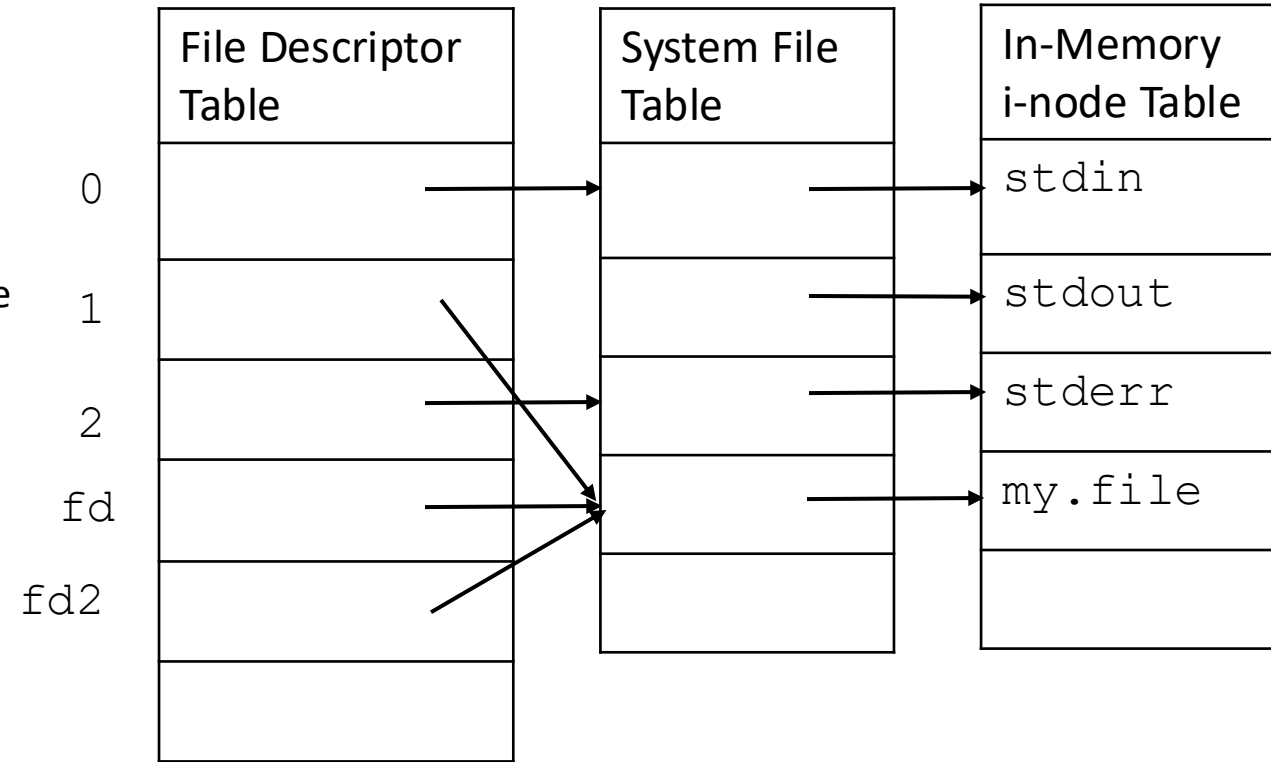
```
    dup(fd); //duplicate the fd-th pointer into entry 1
```

```
    execvp(cmd[0], cmd); //the old process image is replaced by the new process image for ls
```

```
    close(fd); //close file descriptor 3 in the parent process.
```

```
    return;
```

```
}
```



## Demo Task8

# I/O redirection realization using dup ( )

`int dup(int oldfd)` duplicates the given file descriptor to the lowest numbered unused file descriptor in the file descriptor table.

```
#include<stdio.h>
```

```
#include<fcntl.h>
```

```
#include<unistd.h>
```

```
char* cmd[] = {"/bin/ls", "-l", 0};
```

```
int main(int argc, char* argv[]){
```

```
    int fd = open(argv[1], O_WRONLY | O_CREAT, 0600);
```

```
    //fd will be 3; a file will be opened in write mode
```

```
    int fd2 = dup(fd); //duplicate the fd-th pointer to entry 4, the lowest available entry
```

```
    close(STDOUT_FILENO);
```

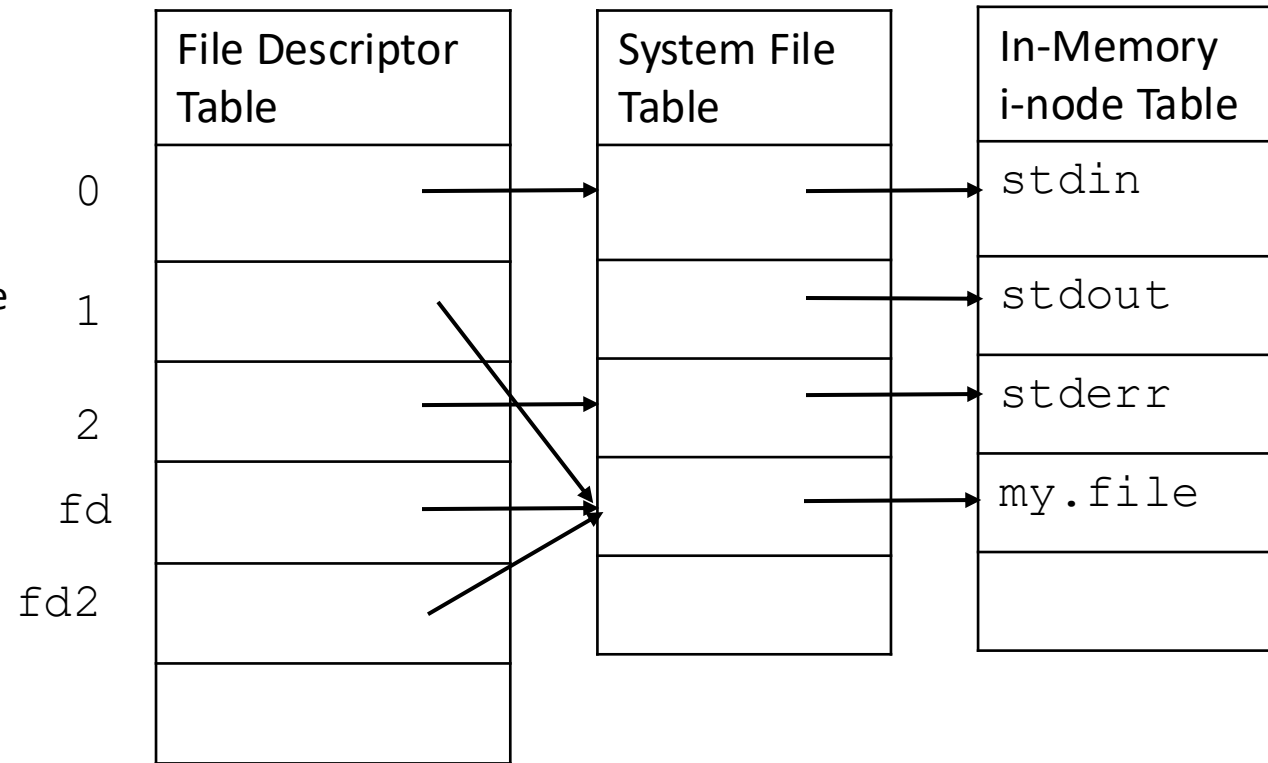
```
    dup(fd); //duplicate the fd-th pointer into entry 1
```

```
    execvp(cmd[0], cmd); //the old process image is replaced by the new process image for ls
```

```
    close(fd); //close file descriptor 3 in the parent process.
```

```
    return;
```

```
}
```



## Demo Task8

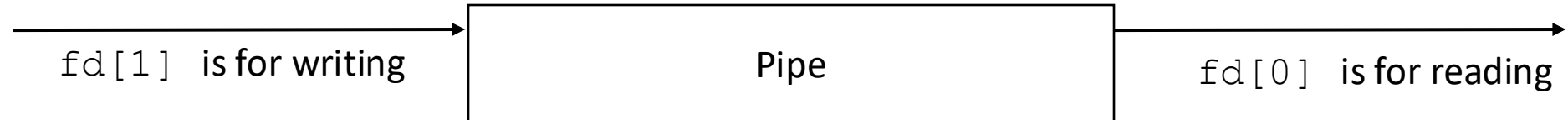
# Communication between parent/child via pipe

System call `pipe()` returns two file descriptors by which we can access the input/output of a pipe (an I/O mechanism)

```
int fd[2];
```

```
int pipe(int fd[2]);
```

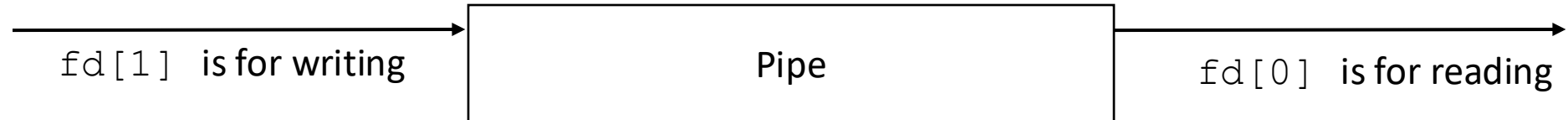
**Return:** 0 success; -1 error



# Communication between parent/child via pipe

## Demo Task9

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <string.h>
int main(){
    int fd[2];
    pipe(fd); //fd[0] is for read and fd[1] is for write
    pid_t child = fork();
    if (child == 0){ //child process
        close(fd[1]);
        char data[100]; //a large enough buffer to share data
        read(fd[0], data, sizeof(data));
        printf("%s\n", data);
        close(fd[0]);
    }else{ //parent process
        close(fd[0]);
        char* data = "hello world";
        write(fd[1], data, strlen(data)+1);
        close(fd[1]);
    }
    return 0;
}
```



# More about Unix files

Details on C library functions/system calls for creating, opening, reading, writing, locating, renaming, deleting, and closing files and directories.